- **Linker**

Other than interpretation and compilation, there is another process known as Linking. Linking is connecting user commands with standard library functions. For example, the program Input and Output commands are linked to the main program during the linking process. For example, when a user writes *printf* the linker links the program with the printing subprogram stored in the Linker.

The following figure depicts the hierarchical position of Hardware, Machine language, Assembly language and High level languages.

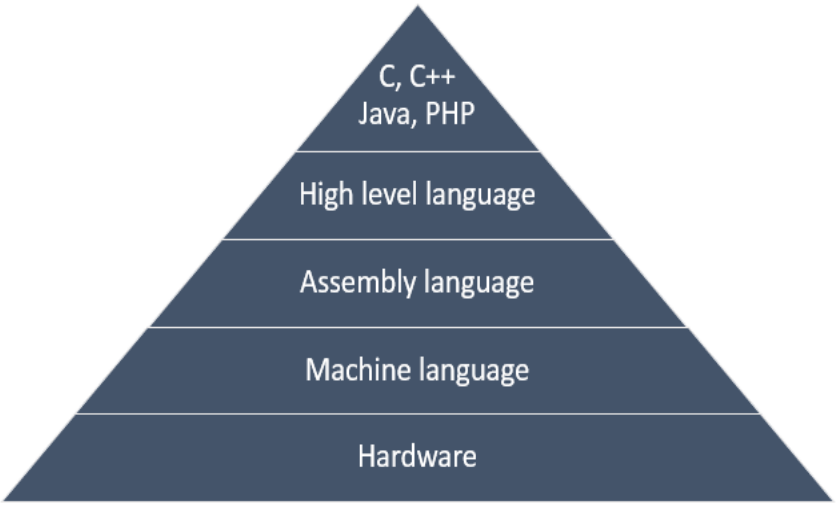


Fig 9.3 Hardware and computer Languages

Competency Level 9.6:
Explores integrated development environment (IDE) to identify their basic features

Learning Outcomes:

- Identifies the basic features of IDE
- Practices the instructions to
  - Open and save files
  - Compile, execute programs
- Uses the debugging facilities in IDE

Integrated development environment (IDE)
Integrated Development Environment is essential to handle the source code. Most IDE's consists code editor, compiler, debugger etc.

As mentioned above, once the source code is written in a high-level language, it is compiled to an object code ready for execution.

Let's see what happens in the process and what programs are responsible for each of these stages.
Editing using Editors – This is the first phase of the process. The algorithm has to be converted to statements according to the syntax (grammar) of the language, and these statements have to be typed for further processing. This process is called Editing and the software that helps to do this work is called Editors. There are various types of Editors providing different features. For example, three main functions of an Editor are creating a new file, saving a file, Opening a saved file etc. Basically, an Editor is word processing software with minimal features.

Debugging/ Compiling using Compilers – After the program editing is over, a source file is created. As mentioned above, this source file has to be converted to an object file by compiling. In the process of compilation, another process called debugging is taking place. This is the process that detects the syntax errors in the source file. Debugging can be done before compiling, but most of the time, debugging and compilation are happening simultaneously. Debugging program is called a Debugger, and the compilation program is called a Compiler.

Nowadays, there are comprehensive programs which are known as Integrated Development Environments (IDE) where editing, debugging/ compiling can be done very convenient way using one comprehensive program containing an Editor, a Debugger and a Compiler. They are very user-friendly and work through user-friendly interfaces.

Microsoft Visual Studio is such an IDE, Eclipse with PyDev plugin is an IDE for development in Python.

There are many online IDEs to develop Python programs.

Imperative programming language to encode algorithms:
In the following sections, programming language Python is used to demonstrate the program structures and syntax. These structures and syntax are almost similar in all the languages. Once the logical programming techniques are mastered, anybody can switch to any other language with little self-work.

- **Structure of a program**
  A program has three main parts, namely, Input, Processing, and Output.
  As we know, Input/ Output are handled using variable names. Logically, before using a variable, it has to be defined. So, we define the program variables, before the Input section.
  As mentioned earlier, a program uses predefined functions like print(), writeln() etc. These are stored in a library, and these library functions have to be loaded at the beginning.
  Accordingly, a structure of a program looks as follows:

```
Start
        Importing essential libraries
        Declaration of variables/ constants
        Input data
        Processing
        Output results
End
```

- **Comment**
  In a program, it is necessary to identify and understand the functions of different lines in an ordinary language understandable to humans. As we know, the program contains the language understandable to the computer, so to bridge the gap, we write comments at the beginning or end of commands. These are not executed by the compiler and the only use of Comments is to understand the function of the statement or the command. Comments are essential to understand the function of the commands/ program by people other than the programmer and even by the programmer at a later time.

  In Python, we could write comments as below.

```
age=26
if (age>=18):
    print ("You are eligible to vote"); # To print the eligibility
```

Competency Level 9.7: Uses an imperative programming language to encode algorithms

Learning Outcomes:

- Identifies the structure of a program
- Uses comments to identify the usage of code for future reference
- Uses constants and variables in a program appropriately
- Identifies the primitive data types of a given program language
- Identifies and uses operators in a program
- Identifies precedence of operators
- Writes programs with the facilities of input from keyboard and output to standard devices

Python program
Python is a popular programming language, which was created in 1991 by Guido van Rossum.

What can Python do?
Python can be used:

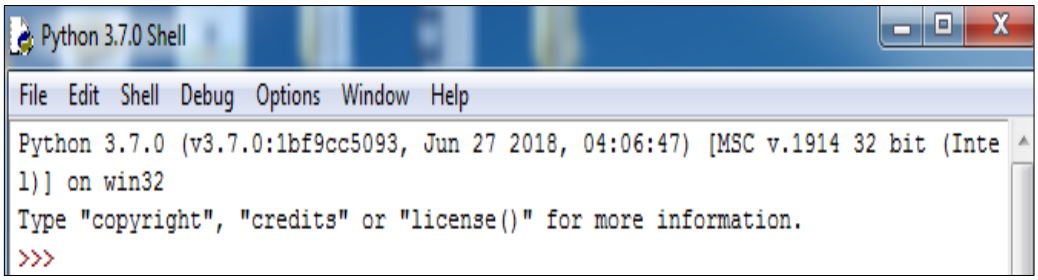
- Alongside software to create workflows.
- To connect to database systems. It can also read and modify files.
- To handle big data and perform complex mathematics.
- For rapid prototyping, or for production-ready software development.
- On a server to create web applications.

Features of Python

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has a syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-orientated way or a functional way.

Python Installation
If you find that you do not have python installed on your computer, you can freely download and install from the following website: https://www.python.org/downloads/

The IDLE of Python (Python 3.7):



Save the python file
Python file should be saved with .py extension.
e.g:- "hello.py"

Let's develop our first Python programin the python shell and python editor.
Write print("My First Program")in python shelland enter

Source

```
*Python 3.7.0 Shell*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27
2018, 04:06:47) [MSC v.1914 32 bit (Int
el)] on win32
Type "copyright", "credits" or "license
()" for more information.
>>> print("My First Program") |
```

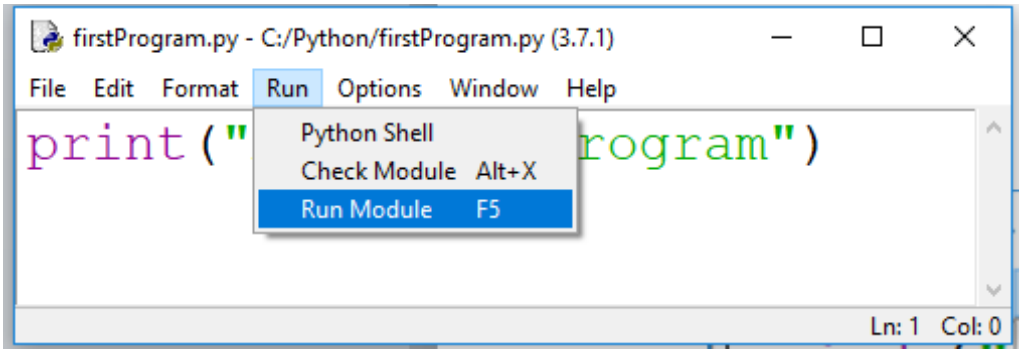
Output

```
Python 3.7.0 Shell
File  Edit  Shell  Debug  Options  Window  Help
Python 3.7.0 (v3.7.0:1bf9cc5093, Jun 27
2018, 04:06:47) [MSC v.1914 32 bit (Int
el)] on win32
Type "copyright", "credits" or "license
()" for more information.
>>> print("My First Program")
My First Program
>>> |
```

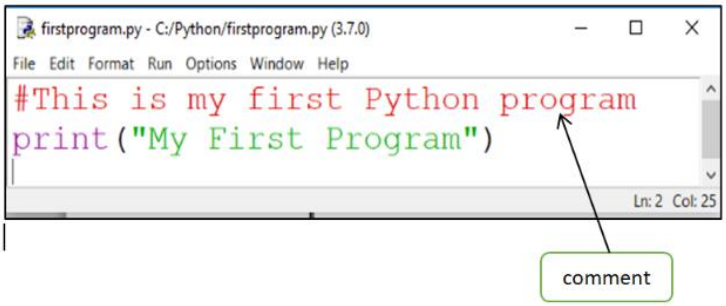
Write print("My First Program")in python editor
Save as FirstProgram1.py
Open the Python editor
File→ New

```
FirstProgram1.py - C:/Python/FirstProgram1.py (3.7.0)
File  Edit  Format  Run  Options  Window  Help
print("My First Program")
|
Ln: 1  Col: 0
```

Run the program
Run →Run Module or press F5 key



Comment in Python
In Python,comments start with a **# symbol**, and Python will render the rest of the line as a comment:



Python Identifier
An identifier is a name that identifies an object (variable, function, class etc). Different languages use different rules for naming objects. In Python the following rules must be observed in naming objects.

- An Identifier name in python must start with a letter or underscore character
- After the first letter, the remainings of an identifier name can contain alpha-numeric characters and underscores and numbers (A-z, 0-9, and _)
- There is no limit to the maximum length of an identifier name
- Python keywords can't be used
- Identifier names are case-sensitive (age, Age, and AGE are three different variables)

Variable
A variable is the symbolic name assign for a place in the computer's memory where one can store data. Once a variable is created by a program, that variable can be used to store different values of the same data type at different times during program execution

Creating Variables in Python
Unlike other programming languages, Python has no command for declaring a variable.
At the moment of creating a variable, the value should be assigned.

Example 1:

Code Output

```
File Edit Format Run Op
a=10
b="Saman"
print(a)
print(b)
```

```
>>>
10
Saman
>>>
```

Example 2:

Code Output

```
Python 2.7.8: ex1.py - D:/Pythion/ex1.py
File Edit Format Run Options Windows Help
a=6         # a is of type int
a="Kamal" # a is now a type str
print (a)
Ln: 2 Col: 31
```

```
>>>
Kamal
>>>
```

Concatenation in python

In combine operation, Python uses the **"+" operator**

Example 3:

Code Output

```
File Edit Format Run Options Windows Help
x = "a programming language"
print("Python is " + x)
```

```
>>>
Python is a programming language
>>>
```

Standard Data types in python

Numbers

- Integral
  - Integer
  - Boolean
- Real
- Complex

Sequences

- Immutable sequences
  - Strings
  - Tuples
  - Bytes
  -
- Mutable sequences
  - Lists
  - Byte arrays

Set types

  - Sets
  - Frozen sets

Mappings

- Dictionaries

Variables of numeric types are created when you assign a value to them:

- x = 1 # int
- y = 2.8 # float
- z = 1j # complex

Specify a data type to a variable

There are situations to specify a type to a variable in a program. This can be done with the casting of the data type. Python uses classes to define data types.

- int() - constructs an integer number from an integer literal, a float literal or a string literal.
- float() - constructs a float number from an integer literal, a float literal or a string literal.
- str() - constructs a string from a wide variety of data types, including strings, integer literals and float literals.

Example 4:
Integer:

Code **Output**

```
a = int(5)
b = int(4.6)
c = int("2")

print("a = ", a)
print("b = ", b)
print("c = ", c)
```

```
>>>
a =  5
b =  4
c =  2
>>>
```

Floats:
Code **Output**

```
a = float(2)
b = float(3.8)
c = float("4")
d = float("5.2")

print('a =',a)
print('b =',b)
print('c =',c)
print('d =',d)
```

```
>>>
a = 2.0
b = 3.8
c = 4.0
d = 5.2
>>>
```

Example 6:
Strings:
Code **Output**

```
a = str("NIE")
b = str(4)
c = str(8.0)

print("a =", a)
print("b =", b)
print("c =", c)
```

```
>>>
a = NIE
b = 4
c = 8.0
>>>
```

String Literals
String literals in Python are surrounded by either single quotation mark or double quotation mark.
'hello' is same as "hello"
There are many string functions available in Python to handle strings literals.

Example 7:

Code Output

```
File Edit Format Run Options Windows Help
a = "Welcome to, Ice Age!" World!"
print(a[1])
print(a[3:5])
print(a.strip())
print(len(a))
print(a.lower())
print(a.upper())
print(a.replace("e", "a"))
print(a.split("  " ,))
```

```
>>>
e
co
Welcome to, Ice Age!" World!
28
welcome to, ice age!" world!
WELCOME TO, ICE AGE!" WORLD!
Walcoma to, Ica Aga!" World!
['Welcome to, Ice Age!" World!']
>>>
```

User Input:

Python allows the user to input data through command line input.

Example 8:

The following example asks for the user's name, by using the **input()** method, the program prints the name on the screen:

Code **Output**

```
File Edit Format Run Options Windows Help
print("Enter your name :")
a = input()
print("Hello, " + a)
```

```
>>>
Enter your name :
kamal
Hello, kamal
>>>
```

Operators in Python

Operators are used to perform operations on variables and values.

Python operators:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Bitwise operators

Arithmetic Operators

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

Example9:

Code **Output**

```
File  Edit  Format  Run  Options  Windows  Help
M1 = 7
M2 = 8
print("M1+M2 =" , M1+M2)
```

```
>>>
M1+M2 = 15
>>>
```

Example 10:
CodeOutput

```
File  Edit  Format  Run  Optic
a = 6
b = "Ravi"
print(a+b)
```

```
>>>
Traceback (most recent call last):
  File "C:\Python33\python program\Pyhon programme\ex11.py", line 3,
in <module>
    print(a+b)
TypeError: unsupported operand type(s) for +: 'int' and 'str'
>>>
```

Note: Integer and string cannot be calculated.Trycalculations using different data types.

Assignment Operators
Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	c= 5	c = 5
+=	m += 3	m = m + 3

Comparison Operators:
Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	a == b
!=	Not equal	a != b
>	Greater than	a > y
<	Less than	b< y
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

Logical Operators:
Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	a < 5 and b< 10
or	Returns True if one of the statements is true	a < 5 or b< 4
not	Reverse the result, returns False if the result is true	not(a < 5 and b< 10)

Bitwise Operators
Bitwise operators are used to compare (binary) numbers:

Operator	Name	Description	Example Assume a=60, b=13
&	AND	Sets each bit to 1 if both bits are 1	(a & b) =12 (means 0000 1100)
|	OR	Sets each bit to 1 if one of two bits is 1	(a | b) = 61 (means 0011 1101)
~	NOT	Inverts all the bits	(~a) = -61 (means 1100 0011 in 2's complement)

Operators Precedence
The following table shows the precedence from the higher-order to the lower.

Operator	Description
**	Exponentiation (raise to the power)
~	Complement
+, -	unary plus and minus (e.g:- x++,x--)
*, / , % ,//	Multiply, divide, modulo and floor division
+, -	Addition and subtraction
>>, <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise `XOR' (exclusive ‘OR’)
|	Bitwise `OR'
<= , <>, >=	Comparison operators
<>,==, !=	Equality operators
= , %=, /=, //=, -=, +=, *=, **=	Assignment operators
is, is not	Identity operators
in, not in	Membership operators
not	Logical operator
and	Logical operator
or	Logical operator